```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("andrewmvd/helmet-detection")

print("Path to dataset files:", path)
```

```
Downloading from https://www.kaggle.com/api/v1/datasets/download/andrewmvd/helmet-detection?dataset_version_number=1...
100%|██████████| 391M/391M [00:10<00:00, 40.9MB/s]Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/andrewmvd/helmet-detection/versions/1
```

```
!mv /root/.cache/kagglehub/datasets/andrewmvd/helmet-detection/versions/1 /content/helmet-detection
```

```
import os
import random
import xml.etree.ElementTree as ET
import cv2
import matplotlib.pyplot as plt

image_dir = "/content/helmet-detection/annotations"
annotation_dir = "/content/helmet-detection/images"
```

```
import os

print("Images folder contents:", os.listdir("/content/helmet-detection/images")[:10])
print("Annotations folder contents:", os.listdir("/content/helmet-detection/annotations")[:10])
```

```
Images folder contents: ['BikesHelmets36.png', 'BikesHelmets214.png', 'BikesHelmets101.png', 'BikesHelmets197.png', 'BikesHelmet
Annotations folder contents: ['BikesHelmets26.xml', 'BikesHelmets469.xml', 'BikesHelmets249.xml', 'BikesHelmets502.xml', 'BikesH
```

```
import os

image_files = os.listdir("/content/helmet-detection/images")
print(image_files[:10])
```

```
['BikesHelmets36.png', 'BikesHelmets214.png', 'BikesHelmets101.png', 'BikesHelmets197.png', 'BikesHelmets546.png', 'BikesHelmets
```

```
import os
import random
import xml.etree.ElementTree as ET
import cv2
import matplotlib.pyplot as plt

# Paths
images_dir = "/content/helmet-detection/images"
annotations_dir = "/content/helmet-detection/annotations"

# List of image files (PNG)
image_files = [f for f in os.listdir(images_dir) if f.endswith('.png')]

# Randomly pick 20 images
sample_images = random.sample(image_files, 20)

def visualize_image_with_boxes(image_name):
    image_path = os.path.join(images_dir, image_name)
    annotation_name = image_name.replace('.png', '.xml')
    annotation_path = os.path.join(annotations_dir, annotation_name)

    # Check if annotation exists
    if not os.path.exists(annotation_path):
        print(f"Annotation file not found: {annotation_path}")
        return

    # Load image
    img = cv2.imread(image_path)
    if img is None:
```

```
            print(f"Failed to load image: {image_path}")
            return
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Parse XML
        tree = ET.parse(annotation_path)
        root = tree.getroot()

        # Draw boxes
        for obj in root.findall('object'):
            bbox = obj.find('bndbox')
            if bbox is None:
                continue
            xmin = int(bbox.find('xmin').text)
            ymin = int(bbox.find('ymin').text)
            xmax = int(bbox.find('xmax').text)
            ymax = int(bbox.find('ymax').text)

            cv2.rectangle(img, (xmin, ymin), (xmax, ymax), (255, 0, 0), 2)
            label = obj.find('name').text
            cv2.putText(img, label, (xmin, ymin-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,0,0), 2)

        plt.figure(figsize=(10, 8))
        plt.imshow(img)
        plt.axis('off')
        plt.title(image_name)
        plt.show()

    # Visualize 20 random images with bounding boxes
    for img_name in sample_images:
        visualize_image_with_boxes(img_name)
```

BikesHelmets391.png

BikesHelmets718.png

```
import shutil

# Create folders if not exist
train_img_dir = "/content/helmet-dataset/train/images"
train_ann_dir = "/content/helmet-dataset/train/annotations"
val_img_dir = "/content/helmet-dataset/val/images"
val_ann_dir = "/content/helmet-dataset/val/annotations"

for d in [train_img_dir, train_ann_dir, val_img_dir, val_ann_dir]:
    os.makedirs(d, exist_ok=True)

# Split ratio
split_ratio = 0.8
num_train = int(len(image_files) * split_ratio)

# Shuffle image files again for consistent split
random.shuffle(image_files)

train_files = image_files[:num_train]
val_files = image_files[num_train:]

# Move or copy files to new folders
for f in train_files:
    shutil.copy(os.path.join(images_dir, f), train_img_dir)
```

```
    ann_file = f.replace('.png', '.xml')
    shutil.copy(os.path.join(annotations_dir, ann_file), train_ann_dir)

for f in val_files:
    shutil.copy(os.path.join(images_dir, f), val_img_dir)
    ann_file = f.replace('.png', '.xml')
    shutil.copy(os.path.join(annotations_dir, ann_file), val_ann_dir)

print(f"Train images: {len(train_files)}, Validation images: {len(val_files)}")
```

```
Train images: 611, Validation images: 153
```

BikesHelmets464.png

```
import os
import xml.etree.ElementTree as ET

annotations_dir = "/content/helmet-detection/annotations"

class_names_found = set()

for filename in os.listdir(annotations_dir):
    if not filename.endswith('.xml'):
        continue
    xml_path = os.path.join(annotations_dir, filename)
    tree = ET.parse(xml_path)
    root = tree.getroot()

    for obj in root.findall('object'):
        class_name = obj.find('name').text
        class_names_found.add(class_name)

print("Classes found in XML files:", class_names_found)
```

```
Classes found in XML files: {'Without Helmet', 'With Helmet'}
```

```
import xml.etree.ElementTree as ET
import os

def convert_xml_to_yolo(xml_file, img_width, img_height, classes_map):
    tree = ET.parse(xml_file)
    root = tree.getroot()

    yolo_annotations = []

    for obj in root.findall('object'):
        cls = obj.find('name').text
        if cls not in classes_map:
            continue  # Skip classes not in your defined classes

        cls_id = classes_map[cls]

        bbox = obj.find('bndbox')
        xmin = float(bbox.find('xmin').text)
        ymin = float(bbox.find('ymin').text)
        xmax = float(bbox.find('xmax').text)
        ymax = float(bbox.find('ymax').text)

        # Convert to YOLO format
        x_center = ((xmin + xmax) / 2) / img_width
        y_center = ((ymin + ymax) / 2) / img_height
        width = (xmax - xmin) / img_width
        height = (ymax - ymin) / img_height

        yolo_annotations.append(f"{cls_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}")

    return yolo_annotations
```

```
import cv2

# Create labels directories
train_label_dir = "/content/helmet-dataset/train/labels"
val_label_dir = "/content/helmet-dataset/val/labels"
os.makedirs(train_label_dir, exist_ok=True)
```

```
os.makedirs(val_label_dir, exist_ok=True)

def convert_folder(img_dir, ann_dir, label_dir, classes_map):
    for img_file in os.listdir(img_dir):
        if not img_file.endswith('.png'):
            continue

        img_path = os.path.join(img_dir, img_file)
        ann_file = img_file.rsplit('.',1)[0] + '.xml'
        ann_path = os.path.join(ann_dir, ann_file)

        if not os.path.exists(ann_path):
            print(f"Annotation not found for {img_file}, skipping")
            continue

        # Read image size
        img = cv2.imread(img_path)
        h, w = img.shape[:2]

        yolo_ann = convert_xml_to_yolo(ann_path, w, h, classes_map)

        # Save YOLO txt file
        label_file = os.path.join(label_dir, img_file.rsplit('.',1)[0] + '.txt')
        with open(label_file, 'w') as f:
            for line in yolo_ann:
                f.write(line + '\n')

# Classes mapping
classes_map = {'Without Helmet': 0, 'With Helmet': 1}

# Convert train and val sets
convert_folder(train_img_dir, train_ann_dir, train_label_dir, classes_map)
convert_folder(val_img_dir, val_ann_dir, val_label_dir, classes_map)

print("Conversion to YOLO format done!")
```

```
Conversion to YOLO format done!
```

```
dataset_yaml = """
train: /content/helmet-dataset/train/images
val: /content/helmet-dataset/val/images

nc: 2

names: ['Without Helmet', 'With Helmet']
"""

yaml_path = "/content/helmet-dataset/helmet_dataset.yaml"
with open(yaml_path, 'w') as f:
    f.write(dataset_yaml)

print(f"YOLOv8 dataset config saved to {yaml_path}")
```

```
YOLOv8 dataset config saved to /content/helmet-dataset/helmet_dataset.yaml
```

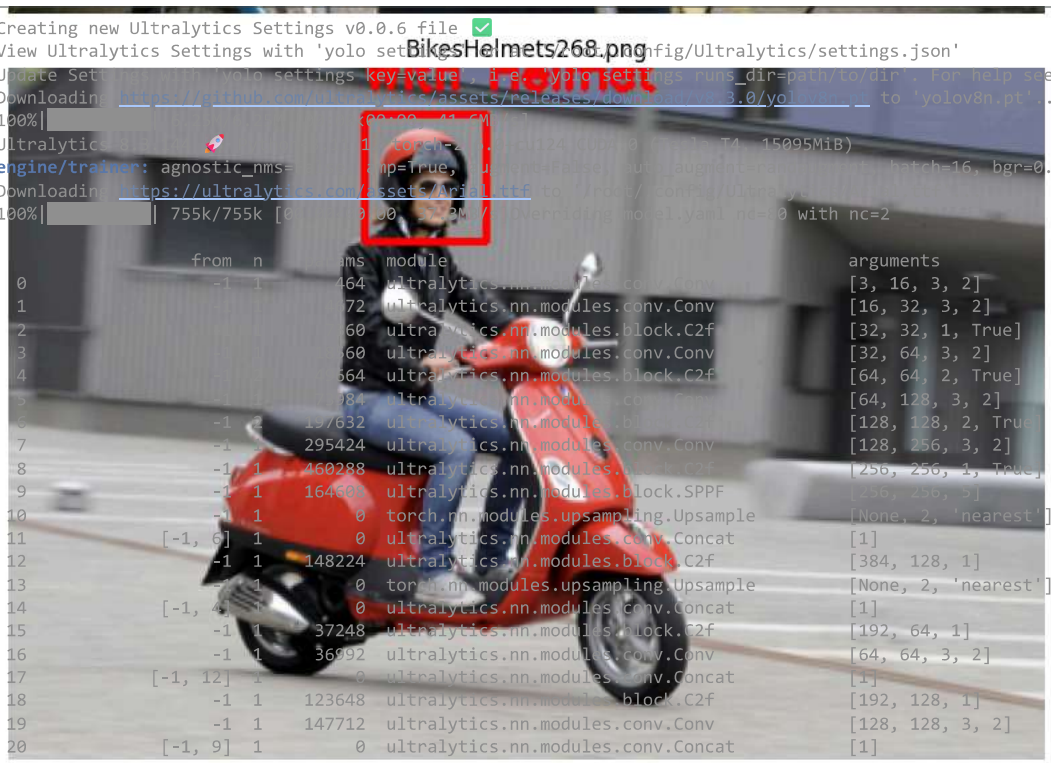
BikesHelmets307.png

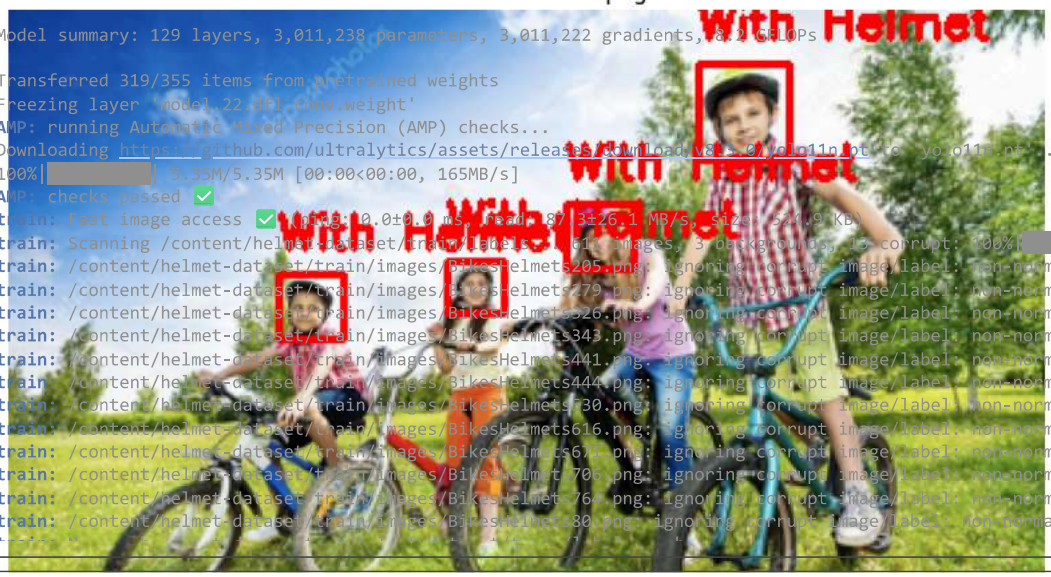
```
!pip install ultralytics --quiet
```

```
from ultralytics import YOLO

# Load a YOLOv8 model - 'yolov8n.pt' is nano (small, fast)
model = YOLO('yolov8n.pt')
```

```
# Train the model
model.train(data='/content/helmet-dataset/helmet_dataset.yaml',
            epochs=30,      # You can increase/decrease this
            imgsz=640,      # Image size
            batch=16,       # Batch size (adjust depending on GPU)
            name='helmet_detection_yolov8')
```

```
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralyti
Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt to 'yolov8n.pt'...
100%|
Ultralytics
engine/trainer: agnostic_nms=
Downloading https://ultralytics.com/assets/Arial.ttf
100%|          | 755k/755k [0
                                                                                                  batch=16, bgr=0.0, box=7.5, cache=False
                                                                                                  Overriding model.yaml nc=80 with nc=2

                 from  n    ms  module                                       arguments
  0                 -1  1   464  ultralytics.nn.modules.conv.Conv            [3, 16, 3, 2]
  1                 -1  1    72  ultralytics.nn.modules.conv.Conv            [16, 32, 3, 2]
  2                 -1  1   640  ultralytics.nn.modules.block.C2f            [32, 32, 1, True]
  3                 -1  1   360  ultralytics.nn.modules.conv.Conv            [32, 64, 3, 2]
  4                 -1  2   564  ultralytics.nn.modules.block.C2f            [64, 64, 2, True]
  5                 -1  1   984  ultralytics.nn.modules.conv.Conv            [64, 128, 3, 2]
  6                 -1  2 197632  ultralytics.nn.modules.block.C2f            [128, 128, 2, True]
  7                 -1  1 295424  ultralytics.nn.modules.conv.Conv            [128, 256, 3, 2]
  8                 -1  1 460288  ultralytics.nn.modules.block.C2f            [256, 256, 1, True]
  9                 -1  1 164608  ultralytics.nn.modules.block.SPPF           [256, 256, 5]
 10                 -1  1      0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 11            [-1, 6]  1      0  ultralytics.nn.modules.conv.Concat           [1]
 12                 -1  1 148224  ultralytics.nn.modules.block.C2f            [384, 128, 1]
 13                 -1  1      0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 14            [-1, 4]  1      0  ultralytics.nn.modules.conv.Concat           [1]
 15                 -1  1  37248  ultralytics.nn.modules.block.C2f            [192, 64, 1]
 16                 -1  1  36992  ultralytics.nn.modules.conv.Conv            [64, 64, 3, 2]
 17           [-1, 12]  1      0  ultralytics.nn.modules.conv.Concat           [1]
 18                 -1  1 123648  ultralytics.nn.modules.block.C2f            [192, 128, 1]
 19                 -1  1 147712  ultralytics.nn.modules.conv.Conv            [128, 128, 3, 2]
 20            [-1, 9]  1      0  ultralytics.nn.modules.conv.Concat           [1]
 21                 -1  1 493056  ultralytics.nn.modules.block.C2f            [384, 256, 1]
 22       [15, 18, 21]  1 751702  ultralytics.nn.modules.head.Detect           [2, [64, 128, 256]]
Model summary: 129 layers, 3,011,238 parameters, 3,011,222 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt to 'yolo11n.pt'...
100%|          | 5.35M/5.35M [00:00<00:00, 165MB/s]
AMP: checks passed ✅
train: Fast image access ✅ (ping: 0.0±0.0 ms, read: 87±126.1 MB/s, size: 54.9 KB)
train: Scanning /content/helmet-dataset/train/labels... 611 images, 3 background, 15 corrupt: 100%|          | 611/611 [00:02
train: /content/helmet-dataset/train/images/BikesHelmets205.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets279.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets326.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets343.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets441.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets444.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets530.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets616.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets674.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets706.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets761.png: ignoring corrupt image/label: non-normalized or out of bounds
train: /content/helmet-dataset/train/images/BikesHelmets801.png: ignoring corrupt image/label: non-normalized or out of bounds
```






```
# Load it to save under a custom name or path:
model = YOLO('runs/detect/helmet_detection_yolov8/weights/best.pt')
model.save('helmet_detection_final.pt')  # Save wherever you want
```

```
# Load the best model weights after training
best_model = YOLO('runs/detect/helmet_detection_yolov8/weights/best.pt')

# Run validation on the val set (defined in your yaml)
metrics = best_model.val()

print(metrics)  # Prints mAP, precision, recall, etc.
```

```
Ultralytics 8.3.224 🚀 Python-3.11.12 torch-2.9.0+cu124 CUDA:0 (Tesla T4, 15095MiB)
Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 3044.6±1393.2 MB/s, size: 486.1 KB)
val: Scanning /content/helmet-dataset/val/labels.cache... 153 images, 0 backgrounds, 2 corrupt: 100%|          | 153/153 [00:0
val: /content/helmet-dataset/val/images/BikesHelmets75.png: ignoring corrupt image/label: non-normalized or out of bounds coor

                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|          | 10/10 [00:04<00:00,
                   all        151        252      0.691      0.827      0.799      0.469
       Without Helmet         50         78      0.599      0.769      0.705      0.365
            With Helmet        109        174      0.783      0.885      0.882      0.573
Speed: 2.9ms preprocess, 9.8ms inference, 2.0ms loss, 3.0ms postprocess per image
Results saved to runs/detect/val
ultralytics.utils.metrics.DetMetrics object with attributes:

ap_class_index: array([0, 1])
box: ultralytics.utils.metrics.Metric object
confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x7d9e5c899710>
curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']
curves_results: [[array([          0,       0.001,       0.002,       0.003,       0.004,       0.005,       0.006,       0.00700
       0.024024,       0.025025,       0.026026,       0.027027,       0.028028,       0.029029,       0.03003,       0.031031,       0.032032,
       0.048048,       0.049049,       0.050050,       0.051051,       0.052052,       0.053053,       0.054054,       0.055055,       0.056056,
       0.072072,       0.073073,       0.074074,       0.075075,       0.076076,       0.077077,       0.078078,       0.079079,       0.08008,
       0.096096,       0.097097,       0.098098,       0.099099,        0.1001,        0.1011,        0.1021,        0.1031,        0.1041,
        0.12012,        0.12112,        0.12212,        0.12312,        0.12412,        0.12513,        0.12613,        0.12713,        0.12813,
        0.14414,        0.14515,        0.14615,        0.14715,        0.14815,        0.14915,        0.15015,        0.15115,        0.15215,
        0.16817,        0.16917,        0.17017,        0.17117,        0.17217,        0.17317,        0.17417,        0.17518,        0.17618,
        0.19219,        0.19319,        0.19419,         0.1952,         0.1962,         0.1972,         0.1982,         0.1992,         0.2002,
        0.21622,        0.21722,        0.21822,        0.21922,        0.22022,        0.22122,        0.22222,        0.22322,        0.22422,
        0.24024,        0.24124,        0.24224,        0.24324,        0.24424,        0.24525,        0.24625,        0.24725,        0.24825,
        0.26426,        0.26527,        0.26627,        0.26727,        0.26827,        0.26927,        0.27027,        0.27127,        0.27227,
        0.28829,        0.28929,        0.29029,        0.29129,        0.29229,        0.29329,        0.29429,         0.2953,         0.2963,
        0.31231,        0.31331,        0.31431,        0.31532,        0.31632,        0.31732,        0.31832,        0.31932,        0.32032,
        0.33634,        0.33734,        0.33834,        0.33934,        0.34034,        0.34134,        0.34234,        0.34334,        0.34434,
        0.36036,        0.36136,        0.36236,        0.36336,        0.36436,        0.36537,        0.36637,        0.36737,        0.36837,
        0.38238,        0.38339,        0.38439,        0.38539,        0.38639,        0.38739,        0.38839,        0.38939,        0.39039,
        0.40841,        0.40941,        0.41041,        0.41141,        0.41241,        0.41341,        0.41441,        0.41542,        0.41642,
        0.43243,        0.43343,        0.43443,        0.43544,        0.43644,        0.43744,        0.43844,        0.43944,        0.44044,
        0.45645,        0.45745,        0.45846,        0.45946,        0.46046,        0.46146,        0.46246,        0.46346,        0.46446,
        0.48048,        0.48148,        0.48248,        0.48348,        0.48448,        0.48549,        0.48649,        0.48749,        0.48849,
        0.50451,        0.50551,        0.50651,        0.50751,        0.50851,        0.50951,        0.51051,        0.51151,        0.51251,
        0.52852,        0.52952,        0.53053,        0.53153,        0.53253,        0.53353,        0.53453,        0.53554,        0.53654,
        0.54855,        0.54955,        0.55455,        0.55556,        0.55656,        0.55756,        0.55856,        0.55956,        0.56056,
        0.57257,        0.57357,        0.57858,        0.57958,        0.58058,        0.58158,        0.58258,        0.58358,        0.58458,
        0.59659,        0.59759,        0.60261,        0.60361,        0.60461,        0.60561,        0.60661,        0.60761,        0.60861,
        0.61962,        0.62062,        0.62663,        0.62763,        0.62863,        0.62963,        0.63063,        0.63163,        0.63263,
        0.64264,        0.64364,        0.65065,        0.65165,        0.65265,        0.65365,        0.65465,        0.65566,        0.65666,
        0.66667,        0.66767,        0.67468,        0.67568,        0.67668,        0.67768,        0.67868,        0.67968,        0.68068,
        0.69969,        0.70069,        0.70770,        0.70870,        0.70970,        0.71071,         0.7027,         0.7037,         0.7047,
        0.71971,        0.72072,        0.72772,        0.72873,        0.72973,        0.73073,        0.72673,        0.72773,        0.72873,
        0.73574,        0.74575,        0.74675,        0.74775,        0.74875,        0.74975,        0.75075,        0.75175,        0.75275,
        0.75976,        0.76977,        0.77077,        0.77177,        0.77277,        0.77377,        0.77477,        0.77578,        0.77678,
        0.78378,        0.79379,        0.79479,        0.79579,        0.79679,        0.79779,        0.7988,         0.7998,         0.8008,
        0.80781,        0.81782,        0.81882,        0.81982,        0.82082,        0.82182,        0.82282,        0.82382,        0.82482,
        0.83183,        0.84184,        0.84284,        0.84384,        0.84484,        0.84585,        0.84685,        0.84785,        0.84885,
        0.85586,        0.86587,        0.86687,        0.86787,        0.86887,        0.86987,        0.87087,        0.87187,        0.87287,
        0.88889,        0.88989,        0.89089,        0.89189,        0.89289,        0.89389,        0.89489,         0.8959,         0.8969,
        0.91291,        0.91391,        0.91491,        0.91592,        0.91692,        0.91792,        0.91892,        0.91992,        0.92092,
```

BikesHelmets337.png

BikesHelmets622.png

```
import cv2
from matplotlib import pyplot as plt

# Load trained model weights
model = YOLO('runs/detect/helmet_detection_yolov8/weights/best.pt')

# Path to your test image
image_path = '/content/images.jpeg'  # change as needed

# Run prediction (returns a list of results)
results = model.predict(source=image_path, conf=0.25)  # conf = confidence threshold

# results[0].boxes.xyxy contains bounding box coordinates
# results[0].boxes.cls contains class indices
# results[0].boxes.conf contains confidence scores

# Visualize prediction on the image using OpenCV
img = cv2.imread(image_path)

# Draw boxes & labels
for box, cls, conf in zip(results[0].boxes.xyxy, results[0].boxes.cls, results[0].boxes.conf):
    x1, y1, x2, y2 = map(int, box)
```

```
        label = model.names[int(cls)]
        conf_text = f'{conf:.2f}'

        # Draw rectangle
        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
        # Put label text
        cv2.putText(img, f'{label} {conf_text}', (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)

    # Convert BGR to RGB for matplotlib display
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(10, 8))
    plt.imshow(img_rgb)
    plt.axis('off')
    plt.show()
```

```
image 1/1 /content/images.jpeg: 448x640 2 Without Helmets, 11.0ms
Speed: 2.3ms preprocess, 11.0ms inference, 2.7ms postprocess per image at shape (1, 3, 448, 640)
```